

PADRÃO DE ARQUITETURA DE DADOS

Azure Data Lake Storage Gen2 + Databricks Lakehouse
(Ambiente Dev)

Organização por Cliente/Produto — MGI / SEGES / CDATA

Versão: 3.0 | Classificação: Uso Interno

Documento Oficial de Arquitetura e Padronização — Revisão Conceitual

Sumário

1. Objetivo

2. Escopo

3. Princípios Arquiteturais

4. Visão Geral da Arquitetura Proposta

[4.1 Contexto Multi-Produto do CDATA](#)

[4.2 Decisão Central: Container por Produto no ADLS, Catálogo por Produto no Databricks](#)

[4.3 Landing como Decisão Arquitetural Explícita](#)

[4.4 Stack Tecnológico](#)

[4.5 Fluxo de Dados por Produto](#)

5. Padronização nas Quatro Dimensões

[5.1 Dimensão Ambiente](#)

[5.2 Dimensão Governança](#)

[5.3 Dimensão Armazenamento](#)

[5.4 Estratégia de Evolução de Schema](#)

[5.5 Retenção Histórica, Time Travel e Reprocessamento](#)

6. Estrutura Proposta para ADLS Gen2

[6.1 Visão Geral: Container por Produto](#)

[6.2 Responsabilidades das Pastas Funcionais](#)

[6.3 Modelo de Acesso e Ownership para Dados Compartilhados \(shared/\)](#)

[6.4 Hierarquia de Diretórios por Container de Produto](#)

7. Estrutura Proposta para Databricks Unity Catalog

[7.1 Por que Catálogo por Produto?](#)

[7.2 Catálogos — Padrão de Nomenclatura](#)

[7.3 Schemas — Padrão de Nomenclatura](#)

[7.4 Estrutura Completa por Produto](#)

[7.5 Objetos dentro dos Schemas](#)

8. Convenções de Nomenclatura

[8.1 Regras Gerais](#)

[8.2 Tabela de Convenções por Objeto](#)

9. Diretrizes de Governança e Acesso

[9.1 Resource Groups por Produto](#)

[9.2 Modelo de Grupos por Produto](#)

[9.3 Regras de Acesso por Camada](#)

[9.4 Service Principals e Key Vault](#)

[9.5 Proteção de Dados Oficiais no ADLS](#)

10. Boas Práticas Operacionais

[10.1 Checklist para Onboarding de Novo Produto](#)

[10.2 Checklist para Onboarding de Nova Fonte de Dados](#)

[10.3 Observabilidade e Monitoramento](#)

[11. Exemplo Prático — Produto SIADS / Domínio Almoxarifado](#)

[11.1 Contexto](#)

[11.2 Estrutura ADLS](#)

[11.3 Estrutura Databricks](#)

[11.4 Fluxo Bronze → Silver → Gold](#)

[11.5 Controle de Acesso — Produto SIADS](#)

[12. Recomendações Finais](#)

[12.1 Governança do Próprio Padrão](#)

[12.2 Fases de Implementação](#)

[12.3 Evolução Tecnológica Recomendada](#)

[Quadro-Resumo Final — Elementos e Padrões](#)

[Anexo A — Resumo Executivo](#)

[Anexo B — Checklist de Validação da Arquitetura v3.0](#)

1. Objetivo

Este documento estabelece o padrão oficial de arquitetura, organização e governança para o ambiente de dados do MGI/SEGES/CGInf, baseado em Azure Data Lake Storage Gen2 (ADLS Gen2) e Databricks Unity Catalog, operando sob o paradigma Lakehouse com camadas Bronze, Silver e Gold.

A versão 3.0 incorpora dois ajustes conceituais fundamentais em relação às versões anteriores:

- A unidade organizacional primária do ADLS Gen2 é o cliente ou produto de negócio — não a função de armazenamento. Containers são criados por produto (pgd, siads, ouvidoria), e as pastas funcionais (landing, raw, curated, serving, shared, logs, checkpoints, sandbox) existentes dentro de cada container de produto.
- A unidade organizacional primária do Databricks Unity Catalog permanece o catálogo por produto, com schemas por camada e domínio — decisão que garante isolamento estrutural de permissões, linhagem e governança.

Adicionalmente, esta versão endereça lacunas identificadas na v2.0: explicitação do papel do Landing como decisão arquitetural formal, especificação do modelo de acesso para dados compartilhados, diretrizes de retenção histórica e reprocessamento com Delta Time Travel, estratégia de evolução de schema para fontes externas instáveis, e definição de papéis formais de Data Steward (curador de dados) e Data Custodian (custodiador de dados).

2. Escopo

Este documento cobre exclusivamente o ambiente de desenvolvimento (DEV). Os clientes/produtos referenciados como exemplos ao longo do documento são PGD, SIADS e Ouvidoria. A estrutura é desenhada para expansão a novos clientes sem refatoração das convenções base.

Em Escopo

- ADLS Gen2: containers por produto, hierarquia de pastas funcionais internas e organização por domínio.
- Databricks Unity Catalog: catálogos por cliente, schemas por camada e domínio.
- Convenções de nomenclatura para todos os objetos do ecossistema.
- Diretrizes de governança, controle de acesso, Resource Groups por produto e rastreabilidade.
- Estratégia de evolução de schema, reprocessamento histórico e Time Travel Delta.
- Boas práticas operacionais e checklist de onboarding de novas fontes.

Fora de Escopo

- ❑ Ambiente de produção (PRD) — citados apenas como referência de expansão futura.
- ❑ Configuração de clusters, políticas de compute e otimização de performance.
- ❑ Integração com ferramentas externas de catalogação de metadados.

3. Princípios Arquiteturais

Os princípios a seguir orientam todas as decisões de design deste padrão. Eles são restrições arquiteturais — não recomendações opcionais — e devem ser respeitados na evolução do ambiente.

Princípio	Descrição	Implicação Prática
Isolamento por produto	Cada cliente/produto possui seu próprio container no ADLS e catálogo no Databricks, garantindo fronteiras de acesso, governança e evolução independentes.	Falha ou refatoração no SIADS não afeta PGD ou Ouvidoria — nem no storage nem no catálogo.
Organização funcional interna	Dentro de cada container de produto, pastas funcionais (landing, raw, curated, serving, shared, logs, checkpoints, sandbox) organizam os dados por propósito.	Políticas de retenção e acesso são aplicadas por pasta funcional dentro do produto.
Progressividade de camadas	Dados fluem da camada Bronze para Gold de forma unidirecional e progressiva. Cada camada eleva a qualidade dos dados, custo de produção e valor analítico.	Não há acesso de consumidores finais a dados Bronze.
Nomenclatura previsível	Todo objeto do ecossistema segue convenções únicas, documentadas e controladas. Nenhum nome é ad hoc.	Qualquer engenheiro consegue localizar um objeto apenas pelo nome ou mesmo identificar o tipo do objeto.
Governança desde a origem	Owner técnico, owner de negócio, Data Steward e Data Custodian são atribuídos no momento da ingestão.	Dado sem owner definido não avança para Silver ou Gold.
Escalabilidade horizontal	Adicionar um novo cliente/produto exige a criação de um novo container ADLS e catálogo Databricks — sem alterar estruturas existentes.	Onboarding de novo produto: processo documentado de menos de 1 hora.
Baixo acoplamento	Catálogos e containers de clientes distintos não referenciam dados uns dos outros diretamente. Dados compartilhados vivem em pasta shared/ de escopo global.	Shared/global/ em container próprio e catálogo de referência cruzada controlado.
Reprodutibilidade	Pipelines são idempotentes: reexecução com as mesmas entradas produz o mesmo resultado. Time Travel Delta suporta re-ingestão e backfill sem risco de duplicação.	Delta Time Travel
Observabilidade nativa	Logs estruturados, métricas de qualidade e rastreabilidade de execução são requisitos de primeira classe.	Toda execução de pipeline registra volume, status e identidade do executor.

4. Visão Geral da Arquitetura Proposta

4.1 Contexto Multiproduto da CDATA

A CDATA serve múltiplos sistemas de informação do governo federal, cada um com características distintas: origens de dados próprias, times de engenharia independentes, contratos de qualidade específicos e consumidores com perfis diferentes. PGD, SIADS e Ouvidoria são exemplos concretos dessa diversidade.

Esse contexto impõe um requisito arquitetural central: a unidade organizacional primária deve ser o produto — tanto no storage quanto no catálogo lógico. Containers ADLS por produto garantem que políticas de acesso Azure (IAM, RBAC, Resource Groups) sejam aplicadas de forma limpa por produto desde a infraestrutura, sem necessidade de filtros por subdiretório.

4.2 Decisão Central: Container por Produto no ADLS, Catálogo por Produto no Databricks

Decisão	<i>O container ADLS representa o cliente/produto. Dentro de cada container, pastas funcionais (landing, raw, bronze, curated, serving, logs, checkpoints, sandbox) organizam os dados por propósito.</i>
Justificativa	<i>Containers são a unidade de controle de acesso, lifecycle e retenção no ADLS Gen2. Usar o container como fronteira de produto garante que permissões Azure (IAM/RBAC), políticas de lifecycle e Resource Groups sejam aplicados de forma limpa por produto, sem filtros por subdiretório. Isso elimina o risco de vazamento de acesso entre produtos e simplifica a auditoria.</i>
Justificativa (Databricks)	<i>O Unity Catalog é a unidade de isolamento mais forte no Databricks. Usar o catálogo como fronteira de produto garante que permissões, linhagem, volumes e políticas de acesso sejam atribuídos de forma limpa por produto. Schemas, como entidades subordinadas ao catálogo, expressam a camada (Bronze/Silver/Gold) e o domínio temático.</i>

Exemplo: siads/bronze/almojarifado/ — container 'siads' = produto, 'bronze' = função, 'almojarifado' = domínio

Exemplo: siads.001_bnz_almojarifado — catálogo 'siads' = produto, schema = Bronze + domínio almojarifado

4.3 Landing e Raw como Decisão Arquitetural Explícita

Decisão	<i>Landing é uma área de recebimento temporário exclusivamente física no ADLS Gen2. Não existe catálogo, schema ou tabela Databricks correspondente à camada Landing. Dados em Landing não são registrados no Unity Catalog. Assim como os dados em Raw, que acomodam os dados brutos sem quaisquer modificações, representam a cópia fiel da origem.</i>
----------------	---

Justificativa	<i>A ausência de representação lógica no Databricks é intencional. A camada de Landing atua exclusivamente como ponto de entrada e desacoplamento das fontes, recebendo e armazenando os dados exatamente como foram entregues, sem qualquer validação ou transformação. E Raw tem como objetivo auditoria, reprocessamento, recuperação de erro e rastreabilidade.</i>
----------------------	---

Responsabilidade do Landing: receber arquivos de sistemas externos. Tem prazo de retenção de 7 dias — após esse período são descartados e o evento registrado no log.

Responsabilidade do Raw: receber arquivos na sua forma bruta. Não existe um prazo de retenção único, deve ser informado pela área de negócio e normalmente é definido por uma combinação de:

- necessidade de auditoria
- estabilidade da origem
- possibilidade de reprocessamento
- custo
- requisitos legais/regulatórios

O conceito de Landing se diferencia de Stage por focar no recebimento e armazenamento inicial temporário dos dados, sem transformações, enquanto Stage tem como objetivo a preparação e processamento. Para este escopo, será utilizada apenas a camada Landing.

4.4 Stack Tecnológico

- ❑ **ADLS Gen2:** Substrato físico de toda a plataforma. Organizado em containers por produto. Dentro de cada container, a hierarquia segue: pasta_funcional/domínio/.
- ❑ **Databricks Unity Catalog:** Camada de governança e abstração lógica. Catálogos por produto, schemas por camada+domínio, tabelas Delta como objeto padrão.
- ❑ **Delta Lake:** Formato transacional que garante ACID, Time Travel, schema enforcement e otimizações de leitura. É obrigatório nas camadas Silver e Gold e constitui o padrão para a camada Bronze, utilizando tabelas Delta externas (External Tables) armazenadas em External Locations gerenciadas pelo Unity Catalog.
- ❑ **Azure Resource Groups:** Um Resource Group por produto agrupa todos os recursos Azure (storage account, service principals, Key Vault secrets, identidades) do produto, facilitando RBAC, billing e auditoria por produto.

4.5 Fluxo de Dados por Produto

Fluxo Conceitual da Arquitetura de Dados



O fluxo é unidirecional dentro de cada container/catálogo de produto. Dados não transitam entre containers de produtos distintos, exceto através da pasta shared/global/ formalmente definida.

A adoção das camadas Landing, Raw e Bronze depende das características e dos requisitos de cada produto. Em cenários de ingestão simples, os dados podem ser persistidos diretamente na camada Bronze. Quando houver necessidade de retenção temporária dos dados de origem ou separação entre recepção e persistência, poderão ser utilizadas as camadas Landing e/ou Raw. A arquitetura apresentada representa o fluxo conceitual completo, não sendo obrigatória a implementação de todas as camadas em todos os projetos.

5. Padronização nas Quatro Dimensões

5.1 Dimensão Ambiente

O ambiente DEV é a única instância ativa neste momento. A estrutura é desenhada para que a expansão para PRD seja mecânica.

5.2 Dimensão Governança

Governança é um atributo transversal, não uma camada separada. Os elementos a seguir são obrigatórios para todo dado que entra na plataforma.

5.2.1 Papéis Formais por Dataset

Cada dataset registrado na plataforma deve ter quatro papéis formalmente atribuídos:

Papel	Responsabilidade	Quem ocupa
Owner Técnico	Responsável pela pipeline, qualidade técnica e SLA de entrega do dado.	Engenheiro de dados do produto
Owner de Negócio	Autoridade institucional sobre o dado: aprova finalidade de uso, responde por obrigações LGPD, autoriza compartilhamento e homologa o aceite final do dado Gold.	Área gestora/negocial do sistema/produto (ex: equipe PGD, equipe SIADS)
Data Steward (Curador)	Execução cotidiana da curadoria: define e mantém CDEs, glossário e dicionário de dados, valida qualidade semântica, propõe promoção Bronze→Silver e submete aceite Gold ao Owner de Negócio. Define política de acesso para aprovação do Owner.	Representante designado da área de negócio
Data Custodian (custodiador)	Custódia técnica da infraestrutura: configuração de permissões ADLS/Unity Catalog, rotação de secrets, aplicação das políticas de retenção, monitoramento de acesso e conformidade técnica com LGPD.	Arquiteto de dados ou engenheiro de plataforma + compliance

A ausência de qualquer um dos quatro papéis bloqueia a promoção do dado de Bronze para Silver.

5.2.2 Classificação de Dados

Todo *dataset* recebe classificação no momento da ingestão. A classificação determina o nível de controle de acesso aplicável.

Classificação corporativa	Descrição	LGPD	Controle de Acesso	Exemplo
Público	Dados liberados para compartilhamento	Sem informações que permitem identificação pessoal (PII)	Leitura ampla, com controle de integridade	Dados abertos, indicadores públicos
Interno	Uso corporativo sem PII	Não pessoal	RBAC padrão, grupos internos por produto	Métricas operacionais, logs técnicos
Restrito	Dados estratégicos ou com PII básico	Dados pessoais	Grupos específicos aprovados	Matrícula, e-mail corporativo, contratos
Confidencial	Dados pessoais sensíveis	Art. 5º II LGPD	Mínimo privilégio, mascaramento e aprovação formal	CPF, endereço, saúde, biometria
Crítico	Regulatório, sigiloso por lei	Alta criticidade	Acesso excepcional, MFA e auditoria completa	Sigilo legal, investigações

* Personally Identifiable Information (Informação Pessoal Identificável)

5.2.3 Metadados de Negócio e *Critical Data Elements* (CDEs)

Todo objeto registrado no Unity Catalog deve ter metadados de negócio completos. O *Data Steward* é responsável por manter esses metadados atualizados.

Metadados obrigatórios por tabela:

- ☐ Descrição funcional: o que o dado representa em linguagem de negócio, não técnica;
- ☐ Domínio de negócio: a qual domínio temático o dado pertence;
- ☐ Frequência de atualização: diária, semanal, sob demanda;
- ☐ Owner técnico, owner de negócio, *Data Steward* e *Data Custodian*;
- ☐ Classificação de dados (Público/Interno/Restrito/Confidencial/Crítico);
- ☐ SLA de disponibilidade (apenas Silver e Gold).

Critical Data Elements (CDEs):

CDEs são as colunas de maior impacto para decisões de negócio ou conformidade regulatória. Para cada CDE, o Data Steward deve registrar no Unity Catalog:

- ☐ Definição de negócio: o que a coluna representa em linguagem não-técnica;
- ☐ Regras de qualidade: valores válidos, faixas aceitáveis, nulos permitidos;
- ☐ Linhagem de negócio: de qual sistema de origem o CDE deriva;
- ☐ Criticidade: a razão pela qual a coluna é um CDE (decisão de negócio, conformidade LGPD, SLA)

Glossário de Negócio:

O *Data Steward* de cada produto mantém um glossário de termos de negócio registrado como Volume no catálogo do produto (vol_glossario_{produto}), versionado e aprovado pelo Owner de Negócio a cada revisão.

5.2.4 Organização por Domínio dentro do Produto

Dentro de cada catálogo de produto, os dados são organizados por domínio temático nos schemas. Domínios representam agrupamentos semânticos de entidades de negócio relacionadas dentro do produto.

- ☐ SIADS: almoxarifado, material de consumo, contratos, patrimônio
- ☐ Ouvidoria: manifestações, protocolos, classificação, atendimentos
- ☐ PGD: metas, entregas, servidores, avaliação

Novos domínios dentro de um produto devem ser aprovados pelo arquiteto responsável antes de serem criados.

5.2.5 Rastreabilidade

- ☐ Toda execução de pipeline registra: data/hora, executor (*service principal* ou usuário), volume processado e *status*

- ❑ Unity Catalog registra linhagem de tabelas automaticamente durante a computação via Databricks
- ❑ Logs de acesso ao ADLS habilitados com retenção mínima de 90 dias em DEV

5.3 Dimensão Armazenamento

Decisão	<i>Ver seção 6 para estrutura completa de containers, pastas funcionais e hierarquia de diretórios.</i>
----------------	---

Justificativa	<i>Containers são a unidade de controle de acesso, lifecycle e billing no ADLS Gen2. Resource Groups por produto simplificam RBAC e auditoria. Detalhamento na seção 6.</i>
----------------------	---

5.4 Estratégia de Evolução de Schema

Fontes externas são instáveis por natureza — sistemas de governo adicionam, renomeiam e removem colunas sem aviso prévio. A plataforma precisa de uma estratégia explícita para absorver essas mudanças sem quebrar pipelines downstream.

5.4.1 Classificação de Mudanças de Schema

Tipo de Mudança	Descrição	Estratégia	Aprovação Necessária
Aditiva (safe)	Nova coluna adicionada pela fonte	Schema evolution automático no Delta (mergeSchema=true em Bronze). Silver absorve na próxima execução após revisão do <i>Data Steward</i> .	<i>Data Steward</i> valida e classifica a nova coluna
Renomeação (breaking)	Coluna renomeada na fonte	Bronze mantém nome original. Silver implementa mapeamento explícito com alias. Versão antiga mantida por 30 dias.	<i>Owner Técnico + Data Steward</i> aprovam o mapeamento
Remoção (breaking)	Coluna removida da fonte	Bronze registra ausência com <i>flag_col_removed</i> . Silver mantém coluna com NULL e alerta. Gold não é afetado enquanto Silver existir.	<i>Owner de Negócio</i> aprova impacto <i>downstream</i>
Mudança de tipo (breaking)	Tipo de dado alterado pela fonte	Bronze registra como string e <i>flag_type_changed</i> . Silver tenta cast com fallback para NULL. Falha de cast gera alerta.	<i>Owner Técnico</i> valida <i>cast</i> antes de Silver

5.4.2 Versionamento de Schema em Bronze

Cada *schema* Bronze registra a versão do *schema* da fonte no momento da ingestão:

- Coluna `_schema_version` adicionada a cada tabela Bronze com hash do *schema* atual
- Mudanças de *schema* geram nova entrada no registro de versões (`vol_schema_versions_{produto}`)
- Pipelines Silver consomem a versão de *schema* explícita para garantir reprodutibilidade de reprocessamento

5.5 Retenção Histórica, Time Travel e Reprocessamento

Para um ambiente de governo sujeito a auditorias, a capacidade de reproduzir o estado de qualquer dado em qualquer ponto do tempo é um requisito crítico, não um diferencial opcional.

5.5.1 Políticas de Time Travel por Camada

Camada	Retenção de Versões Delta	Retenção Física (ADLS)	Justificativa
Bronze (bronze/)	Bronze usa tabelas Delta external mapeadas sobre arquivos parquet imutáveis em bronze/. Arquivos físicos mantidos como imutáveis após ingestão.	Mínimo de * mês/ ano em DEV.	parquet em bronze é o arquivo histórico imutável. Reprocessamento parte sempre do Raw original.
Silver (curated/)	Sugestão * dias	Arquivos retidos por 90* dias após remoção de versão.	Permite auditar o estado do dado em qualquer ponto dos últimos * dias sem reprocessar Bronze.
Gold (serving/)	sugestão * dias	Arquivos Delta retidos por * dias.	Produtos de dados Gold são consumidos por BI e APIs — auditores podem questionar valores históricos de dashboards.

*Prazos de retenção devem ser definidos junto a área de negócio de cada produto

5.5.2 Estratégia de Re-ingestão Histórica (Backfill)

Backfill é o reprocessamento de um intervalo histórico completo de dados. O processo padrão é:

1. Identificar o intervalo histórico: data de início e fim do período a reprocessar.
2. Verificar disponibilidade dos arquivos Bronze originais em `raw/{produto}/{dominio}/{ano}/{mes}/{dia}/`.
3. Executar pipeline de ingestão em modo backfill com flag `--backfill=true`, que desativa deduplicação por data e ativa idempotência por hash de linha.
4. Silver reprocessa apenas o intervalo afetado usando `MERGE INTO Delta` com chave composta (`id + dt_referencia`).
5. Gold é re-materializado após Silver para garantir consistência.
6. Cada execução de backfill gera entrada específica no log com `run_type=backfill`, intervalo, volume e executor.

5.5.3 Re-ingestão de Emergência

Em caso de corrupção ou perda de dados Silver/Gold, o procedimento é:

- ☐ Silver: restaurar para versão anterior via `RESTORE TABLE siads.002_slv_material_consumo.itens TO VERSION AS OF {n}`
- ☐ Gold: idem com `RESTORE TABLE`, ou re-materializar a partir do Silver restaurado
- ☐ Bronze: imutável por definição — nunca é restaurado, apenas relido
- ☐ Toda operação `RESTORE` é registrada no log de auditoria com aprovação formal do *Data Custodian*

6. Estrutura Proposta para ADLS Gen2

6.1 Visão Geral: Container por Produto

A plataforma utiliza um *container* ADLS por produto/cliente. Dentro de cada container, oito pastas funcionais organizam os dados por propósito. Essa estrutura permite que políticas Azure de acesso, lifecycle e retenção sejam aplicadas por produto através de Resource Groups dedicados.

Container (produto)	Pastas funcionais internas	Resource Group Azure
pgd	landing/, raw/, bronze/, curated/, serving/, logs/, checkpoints/, sandbox/	rg-cdata-pgd
siads	landing/, raw/, bronze/, curated/, serving/, logs/, checkpoints/, sandbox/	rg-cdata-siads
ouvidoria	landing/, raw/, bronze/, curated/, serving/, logs/, checkpoints/, sandbox/	rg-cdata-ouvidoria
[novo_produto]	Mesma estrutura — onboarding sem alteração de estruturas existentes	rg-cdata-{produto}

6.2 Responsabilidades das Pastas Funcionais

Pasta	Função	Justificativa
landing/	Área temporária de chegada. Recebe arquivos externos antes da validação de formato e integridade. Sem representação no Unity Catalog.	Quarentena antes de registrar dados no catálogo. Retenção máxima 7 dias — dados não promovidos são descartados com log.
raw/	Fonte da Bronze — dados brutos exatamente como recebidos da fonte. Imutável após ingestão.	Arquivo histórico da plataforma. Política de retenção mínima de *mês/ ano.
bronze/	Dados brutos persistidos em formato Delta com enriquecimento técnico mínimo (metadados de ingestão, origem, hash, data de carga, nome do arquivo, rastreabilidade). Sem aplicação de regras de negócio.	Camada inicial do Lakehouse para processamento. Mantém rastreabilidade, auditoria, padronização técnica e suporte a reproprocessamento. Delta Time Travel habilitado conforme política de retenção.

Pasta	Função	Justificativa
curated/	Silver — dados tratados e padronizados em formato Delta.	Zona de trabalho analítico principal. Delta com Time Travel habilitado.
serving/	Gold — produtos de dados finais prontos para BI e APIs.	Delta otimizado para leitura. Time Travel com retenção estendida para auditoria.
logs/	Logs operacionais de execução de pipelines, auditorias e relatórios de qualidade.	Política de retenção independente. Retenção mínima 90 dias.
checkpoints/	Estado de Spark Structured Streaming para tolerância a falhas e retomada.	Um diretório por job de streaming.
sandbox/	Experimentação livre. Isolado de dados oficiais. Sem SLA. Sem dados confidenciais ou críticos.	Espaço seguro para exploração sem risco de contaminação de dados oficiais.

*Rever prazos após a entrada em vigor do ambiente de produção

6.3 Modelo de Acesso e Ownership para Dados Compartilhados (shared/)

A área shared/, stored dedidado para dados compartilhados, armazena entidades de referência corporativas reutilizadas por múltiplos produtos e domínios da plataforma. Essas referências são registradas no Unity Catalog em catálogos e schemas dedicados (000_ref_*) e consumidas diretamente pelos demais catálogos, garantindo padronização semântica, reutilização e governança centralizada.

6.3.1 Estrutura de shared/

```
shared/
  global/                <- referências usadas por todos os domínios de produtos
    municipios/
    unidades_federativas/
    orgaos/

  {contexto_ou_dominio}/  <- referências específicas de um contexto
    codigos_materiais/
    classificacoes_contabeis/
```

6.3.2 Quem pode escrever em shared/

A escrita em shared/ é controlada e restrita a service principals e grupos autorizados, garantindo integridade e governança das referências compartilhadas.

Escopo	Quem pode escrever	Aprovação necessária
shared/global/	Service principal ou grupo administrador de referências	Arquiteto de dados e Engenheiro de dados
shared/{contexto}/	Service principal responsável pelo domínio e engenheiros de dados	Arquiteto de dados e Engenheiro de dados

Escopo	Quem pode escrever	Aprovação necessária
Sandbox	Usuário owner do espaço	Não se aplica

6.3.3 Aprovação para novas referências em shared/global/

A criação de uma nova tabela de referência global segue o processo:

1. Arquiteto de Dados aprova a criação e define o *schema*.
2. *Data Custodian* (que pode ser o arquiteto de dados) configura as permissões no Unity Catalog e no ADLS.

6.3.4 Linhagem de Dados Compartilhados no Unity Catalog

Dados em shared/ devem ser registrados no Unity Catalog como tabelas Delta em catálogo corporativo dedicado, utilizando schemas padronizados 000_ref_*::

- Exemplos: shared.000_ref_global, shared.000_ref_geografia, shared.000_ref_governo
- Prefixo 000 garante que referências apareçam antes das camadas Bronze/Silver/Gold na navegação
- O Unity Catalog registra automaticamente a linhagem entre tabelas consumidoras e referências compartilhadas, permitindo rastreabilidade, análise de impacto e identificação de dependências entre produtos e domínios da plataforma.

6.4 Hierarquia de Diretórios por Container de Produto

A hierarquia dentro de cada pasta funcional segue o padrão: {dominio}/{particao_opcional}/. O produto já está implícito no container.

Container: siads/landing/

Padrão: {dominio}/{carga_YYYYMMDD_seq}/

```
siads/landing/  
  almoxarifado/  
    carga_20240115_001/  
      almoxarifado_20240115_001.parquet  
  contratos/  
    carga_20240115_001/  
      contratos_20240115_001.json
```

Container: siads/raw/

Padrão: {dominio}/{ano}/{mes}/{dia}/

```
siads/raw/
```

```
almostrarifado/  
  2024/01/15/  
    almostrarifado_20240115_001.parquet  
contratos/  
  2024/01/15/
```

Container: siads/bronze/

Padrão: {dominio}/{ano}/{mes}/{dia}/

```
siads/bronze/  
  almostrarifado/  
    2024/01/15/  
      almostrarifado_20240115_001.parquet  
contratos/  
  2024/01/15/
```

Container: siads/curated/

Padrão: {dominio}/ (tabela Delta)

```
siads/curated/  
  material_consumo/      <- tabela Delta  
contratos/  
patrimonio/
```

Container: siads/serving/

Padrão: {caso_uso}/

```
siads/serving/  
  indicadores_almostrarifado/      <- produto Gold  
painel_contratos/
```

Containers: siads/logs/ e siads/checkpoints/

```
siads/logs/  
  carga_almostrarifado/  
    2024/01/15/  
      run_20240115_001.json  
siads/checkpoints/  
  stream_almostrarifado/      <- um diretório por job de streaming
```

Container: siads/sandbox/

```
siads/sandbox/  
  squad_dados/  
joao_silva/
```

Os mesmos padrões se aplicam aos containers pgd/ e ouvidoria/ com seus respectivos domínios.

7. Estrutura Proposta para Databricks Unity Catalog

7.1 Por que Catálogo por Produto?

A versão anterior propunha catálogos por camada. Essa abordagem possui algumas limitações:

- ❑ **Isolamento fraco:** permissões no Unity Catalog são configuradas no nível do catálogo. Com catálogos por camada, conceder acesso Silver ao time do SIADS exige filtros explícitos para excluir schemas da Ouvidoria — frágil e propenso a erros.
- ❑ **Acoplamento de ciclo de vida:** Bronze do SIADS e da Ouvidoria compartilham catálogo, mas possuem frequências e owners distintos.
- ❑ **Escalabilidade limitada:** a cada novo produto, schemas precisam ser criados em três catálogos distintos.
- ❑ **Dificuldade de onboarding:** engenheiro do SIADS navega em schemas de outros produtos, gerando risco operacional.

Decisão	<i>Catálogo = cliente/produto. Schema = camada + domínio temático. Referências globais recebem schema 000_ref_*</i>
----------------	---

7.2 Catálogos — Padrão de Nomenclatura

Padrão: {produto}

Catálogo	Produto	Descrição
pgd	PGD	Programa de Gestão e Desempenho
siads	SIADS	Sistema Integrado de Administração de Serviços
ouvidoria	Ouvidoria	Sistema de Ouvidoria do órgão

7.3 Schemas — Padrão de Nomenclatura

Padrão: {prefixo_camada}_{dominio_tematico}

Prefixo	Camada	Padrão do Schema	Exemplos
000	Referências	000_ref_{escopo}	000_ref_global, 000_ref_cep
001	Bronze	001_bnz_{sistema_origem}	001_bnz_siafem, 001_bnz_api_rfb
002	Silver	002_slv_{dominio_tematico}	002_slv_material_consumo, 002_slv_contratos
003	Gold	003_gld_{produto_dados}	003_gld_indicadores, 003_gld_paineis_executivos
999	Sandbox	999_sbx_{produto}	999_sbx_petrvs

A arquitetura adotada dentro do catálogo (Produto) segue o modelo medalhão, estruturado nas camadas Bronze, Silver e Gold.

7.3.1 Referências — Entidades Compartilhadas e Padronização Global

Atributo	Definição
Objetivo	Centralizar entidades de referência compartilhadas entre múltiplos catálogos da plataforma, garantindo padronização semântica, reutilização e consistência institucional.
Transformações	Normalização controlada, padronização de códigos institucionais, consolidação de tabelas mestres, enriquecimento validado e versionamento de entidades de referência.
Consumidor	As camadas da plataforma (Silver e Gold), pipelines de integração, APIs, dashboards e produtos analíticos corporativos.
Qualidade	Muito alta — dados estáveis, amplamente reutilizados, com governança formal, controle de versionamento e ownership claramente definido.
Permitido	Catálogos corporativos, dimensões compartilhadas, tabelas mestres, taxonomias, classificações oficiais, dicionários institucionais e entidades globais reutilizáveis.
Proibido	Duplicação de entidades já existentes, regras específicas de produto, dados transacionais, KPIs, agregações analíticas ou dados temporários de processamento.

7.3.2 Bronze — Fidelidade à Fonte

Atributo	Definição
Objetivo	Preservar o dado original . A camada Bronze é o arquivo histórico da plataforma — imutável após ingestão.
Transformações	Apenas metadados técnicos adicionados: data de ingestão, nome do arquivo, hash de linha, id de execução. Conteúdo original intocável.
Consumidor	Exclusivamente pipelines de transformação Silver. Analistas não acessam Bronze.
Qualidade	Baixa por definição — reflete a realidade da fonte, com duplicatas, nulos e inconsistências possíveis.
Proibido	Correções de negócio, deduplicação, enriquecimento, exclusão de registros originais.

7.3.3 Silver — Verdade Única do Dado

Atributo	Definição
Objetivo	Produzir dados limpos, confiáveis e semanticamente corretos. É a camada de referência para consumo técnico.
Transformações	Limpeza, deduplicação, tipagem correta, padronização de colunas (snake_case), aplicação de regras de qualidade documentadas.
Consumidor	Analistas de dados, cientistas de dados, pipelines Gold.

Atributo	Definição
Qualidade	Alta — schema enforced, sem duplicatas, tipos corretos, nulos tratados conforme regra documentada.
Proibido	KPIs, agregações de negócio, joins especulativos não validados. Lógica de negócio é responsabilidade da camada Gold.

7.3.4 Gold — Produto de Dados

Atributo	Definição
Objetivo	Entregar dados prontos para consumo por ferramentas de BI, APIs e relatórios. Máxima curação, confiabilidade e documentação.
Transformações	Agregações, KPIs, joins multi-domínio, modelos dimensionais, enriquecimento com dados validados.
Consumidor	Power BI, APIs públicas, dashboards executivos, squads de produto, cientistas de dados avançados.
Qualidade	Máxima — SLA definido, testes automatizados, documentação completa, aprovação formal do owner de negócio.
Proibido	Dados brutos não processados, schemas instáveis, objetos sem owner de negócio definido.

7.4 Estrutura Completa por Produto

Catálogo: siads

```

siads
├── 001_bnz_siafem          <- Bronze: dados brutos do SIAFEM
│   ├── almoxarifado
│   └── contratos
├── 001_bnz_planilhas      <- Bronze: uploads manuais
│   └── ajuste_estoque
├── 002_slv_material_consumo <- Silver: entidade consolidada
│   ├── itens
│   ├── movimentacoes
│   └── categorias
├── 002_slv_contratos
│   ├── contratos
│   └── aditivos
└── 003_gld_indicadores
    ├── painel_almox
    └── vw_estoque_atual
  
```

7.5 Objetos dentro dos Schemas

- ❑ **Tabelas Delta (external):** Usadas em Bronze para mapear dados já existentes em raw/ do ADLS, e padrão para Silver e Gold. Gerenciadas pelo Unity Catalog com linhagem automática.
- ❑ **Views (vw_*):** Criadas em Gold para expor subconjuntos com filtros de acesso ou projeções específicas.
- ❑ **Volumes:** Para dados não-tabulares (PDFs, XMLs) com governança Unity Catalog. Preferencialmente em Bronze.
- ❑ **Glossário (vol_glossario_{produto}):** Volume dedicado ao glossário de negócio do produto, mantido pelo Data Steward.
- ❑ **Registro de versões de schema (vol_schema_versions_{produto}):** Histórico de mudanças de schema por fonte.

8. Convenções de Nomenclatura

8.1 Regras Gerais

- ❑ Somente letras minúsculas (a-z), dígitos (0-9) e underscore (_).
- ❑ Proibido: espaços, hífen, acentos, cedilha, caracteres especiais.
- ❑ Máximo 64 caracteres para nomes de objetos Databricks; 120 para caminhos ADLS.
- ❑ Siglas em minúsculo integradas ao nome normalmente (ex: pgd, siads, bi, api).
- ❑ Ambiente identificado por sufixo _hml, _prd nos catálogos quando múltiplos ambientes coexistirem.

8.2 Tabela de Convenções por Objeto

Objeto	Padrão	Exemplo	Observação
Container ADLS	{produto}	siads, pgd, ouvidoria	Um container por produto
Pasta funcional (ADLS nível 1)	{funcao}	landing, raw, bronze, curated, serving, shared, logs, checkpoints, sandbox	9 pastas fixas por container
Dir. nível 2 (ADLS)	{dominio}	almoxarifado, metas	Domínio temático
Catálogo Databricks	{produto}	siads, pgd	Produto em minúsculo
Schema Referências	000_ref_{escopo}	000_ref_global	Referências
Schema Bronze	001_bnz_{sistema_origem}	001_bnz_siafem	Por sistema de origem
Schema Silver	002_slv_{dominio}	002_slv_material_consumo	Por entidade/domínio

Objeto	Padrão	Exemplo	Observação
Schema Gold	003_gld_{produto_dados}	003_gld_indicadores	Por produto de dados
Shema Sandbox	999_sbx_{produto}	999_sbx_petrvs	Por produto de dados
Tabela	{entidade}	manifestacoes, contratos	Snake_case, sem prefixo
View	vw_{entidade}_{escopo}	vw_estoque_atual	Prefixo vw_ obrigatório
Volume	vol_{tipo}_{conteudo}	vol_glossario_siads	Prefixo vol_
Pipeline / Job	{produto}_ {dominio<opicional>} {acao}_job	siads_almoxt_ingest_job	Ação: ingest/transform/publish
Notebook	{produto}_ {dominio<opicional>} {acao}	siads_materiais_transform	Sem numeração forçada
Checkpoint	chk_{job_name}	chk_stream_siads_almoxt	Um por job streaming
Arquivo ingestão	{entidade}_{YYYYMMDD}_{seq}.{ext}	almoxt_20240115_001.parquet	Data+seq = unicidade
Service Principal	sp_{produto}_{acao}	sp_siads_ingest	Um por produto/ação
Grupo de Acesso	grp_{papel}_{produto}	grp_data_analysts_siads	Grupos por papel + produto
Resource Group Azure	rg-cdata-{produto}	rg-cdata-siads	Um por produto por ambiente
Storage Account	st<coordenação><produto>	stcdatasiads	Um por produto
Key Vault	kv-<coordenação>-<produto>	kv-cdata-petrvs	Um por produto
Access Connector	ac-<produto>	ac-petrvs	Um por produto

9. Diretrizes de Governança e Acesso

O modelo de acesso adota grupos do Azure Active Directory (AAD) mapeados a permissões no Unity Catalog e no ADLS Gen2, complementados por Resource Groups Azure por produto. A granularidade do controle é por produto — cada produto possui seus próprios grupos, Resource Group e service principals.

9.1 Resource Groups por Produto

Decisão	<i>Um Resource Group Azure por produto por ambiente (rg-cdata-{produto}). Todos os recursos Azure do produto — storage account, identidades de service principals, Key Vault secrets, políticas de acesso — são agrupados no Resource Group do produto.</i>
----------------	---

Justificativa	<i>Resource Groups são a unidade de billing, RBAC e auditoria no Azure. Agrupar recursos por produto permite: (1) aplicar políticas Azure Policy por produto; (2) auditar custos por produto; (3) conceder acesso de emergência ao Resource Group do produto sem expor outros produtos; (4) facilitar offboarding de produto — deletar o Resource Group remove todos os recursos associados de forma controlada.</i>
----------------------	--

Resource Group	Produto	Recursos incluídos
rg-cdata-pgd	PGD	Storage account (container pgd/), service principals sp_pgd_*, secrets no Key Vault, grupos AAD grp_*_pgd
rg-cdata-siads	SIADS	Storage account (container siads/), service principals sp_siads_*, secrets no Key Vault, grupos AAD grp_*_siads
rg-cdata-ouvidoria	Ouvidoria	Storage account (container ouvidoria/), service principals sp_ouvidoria_*, secrets no Key Vault, grupos AAD grp_*_ouvidoria

9.2 Modelo de Grupos por Produto

Padrão de nomenclatura de grupos: grp_{papel}_{produto}

Grupo AAD	Permissão Unity Catalog	Escopo
grp_data_engineers_{produto}	USE + READ + WRITE + CREATE	Criação e manutenção de objetos nas camadas Bronze, Silver e Gold do produto em ambiente DEV. Leitura em 000_ref_*.
grp_data_architects	Catalog Admin	Administração completa de catálogos, schemas, permissões e padrões arquiteturais.
grp_data_analysts_{produto}	READ (Silver, Gold) + WRITE Sandbox	Consumo analítico em Silver e Gold. Escrita apenas em sandbox própria.
grp_data_scientists	READ (todos os schemas Silver e Gold))	Leitura em todos os schemas Silver e Gold. Escrita na sandbox própria.
grp_data_stewards_{produto}	READ (todos os schemas)	Leitura em todos os schemas do produto para curadoria de metadados. Sem escrita em dados.
sp_{produto}_ingest	READ + WRITE (Bronze)	Service principal de ingestão. Acesso mínimo a landing/, raw/ e schema Bronze do produto.
sp_{produto}_transform	READ (Bronze) + WRITE (Silver)	Service principal de transformação Silver.

Grupo AAD	Permissão Unity Catalog	Escopo
sp_{produto}_publish	READ (Silver) + WRITE (Gold)	Service principal de publicação Gold.
sp_{produto}_shared_write	WRITE (000_ref_*)	Service principal dedicado para escrita em shared/. Aprovação de Arquiteto + Data Steward obrigatória para uso.

9.3 Regras de Acesso por Camada

- ❑ **Landing:** Zona de entrada gerenciada pelo sp_{produto}_ingest. Sem representação no Unity Catalog — acesso restrito ao time técnico do produto.
- ❑ **Raw:** Acessível para leitura pelos engenheiros do produto e leitura e escrita pelos SPs de ingestão.
- ❑ **Bronze:** Acessível para leitura e escrita pelos engenheiros do produto e pelos SPs de ingestão e transformação. Em DEV, permite testes de pipeline, validação de schema e evolução de estruturas.
- ❑ **Silver:** Escrita via engenheiros e SP de transformação. Leitura aberta a analistas, cientistas e arquitetos — em DEV, o acesso pode ser liberado conforme a necessidade da equipe.
- ❑ **Gold:** Escrita via engenheiros e SP de publicação. Em DEV, serve também para validar modelos analíticos e produtos de dados antes de promover para produção.
- ❑ **Catálogo de referências compartilhadas:** Escrita via sp_shared_write com aprovação técnica do Arquiteto de dados. Produtos consomem as referências diretamente do catálogo shared — sem replicar dados nos próprios containers ou catálogos. Em DEV, o processo formal aplica-se a partir do momento em que a referência é consumida por pipelines Silver.
- ❑ **Sandbox:** Espaço livre para experimentação individual. Dados confidenciais, críticos ou regulados não devem ser copiados para sandbox. Somente datasets com Owner Técnico e Owner de Negócio formalmente definidos podem ser copiados para sandbox — datasets sem ownership atribuído permanecem restritos a Bronze.

9.4 Recursos por Produto: Service Principals, Storage Account, Key Vault e Access Connector

- Um service principal por operação por produto (ingest, transform, publish, shared_write).
- Um storage account por produto.

- Credenciais armazenadas no Azure Key Vault do Resource Group do produto — nunca em código ou notebooks.
- Rotação de secrets a cada 180 dias em DEV.

Service principals não devem acessar recursos de outros produtos.

Exceções:

- Recursos corporativos compartilhados da plataforma;
- Catálogo de referências compartilhadas;
- Outros ativos explicitamente autorizados pela Governança de Dados.

Nesses casos, o acesso deve ser concedido apenas com as permissões mínimas necessárias, preferencialmente somente leitura.

9.5 Proteção de Dados Oficiais no ADLS

- ☐ Pastas ADLS: deleção manual não bloqueada para usuários — por se tratar de ambiente de desenvolvimento.
- ☐ Soft delete habilitado no ADLS com retenção de 7 dias em DEV.
- ☐ Versioning habilitado nas pastas curated/ e serving/.
- ☐ Auditoria de acesso habilitada em todos os containers.

10. Boas Práticas Operacionais

10.1 Checklist para Onboarding de Novo Produto

#	Item	Responsável
01	Produto aprovado pelo comitê de arquitetura e nome curto definido (ex: siads, pgd)	Arquiteto de Dados
02	Resource Group Azure criado: rg-cdata-{produto}	Data Custodian + Arquiteto de Dados
03	Service principals criados (ingest, transform, publish, shared_write) e secrets gerados para cada SP	Arquiteto de Dados
04	Key Vault criado	Arquiteto de Dados
05	Secrets armazenados no Key Vault	Arquiteto de Dados
06	Storage Account criado com Hierarchical Namespace habilitado	Arquiteto de Dados
07	Container ADLS criado com as 8 pastas funcionais internas	Arquiteto de Dados
08	Permissões ADLS atribuídas via IAM	Arquiteto de Dados

#	Item	Responsável
	No Databricks	
09	Storage Credential criada	Arquiteto de Dados
10	External Location criada	Arquiteto de Dados
11	Catálogo Databricks criado e owner técnico registrado (como grupo e não pessoa física)	Arquiteto de Dados
12	Grupos AAD criados para o produto (architects, engineers, analysts, stewards)	Arquiteto de Dados
13	Grants aplicados para todos os grupos humanos	Arquiteto de Dados
14	Grants aplicados para todos os Service Principals	Arquiteto de Dados
	Documentação	
15	Papéis formais atribuídos: Owner Técnico, Owner de Negócio, Data Steward, Data Custodian	Arquiteto + Negócio
16	Runbook do produto iniciado (fontes, frequência, owners, pontos de contato)	Engenheiro de Dados
17	Time do produto treinado no padrão de nomenclatura e no fluxo Bronze→Silver→Gold	Arquiteto de Dados

10.2 Checklist para Onboarding de Nova Fonte de Dados

#	Item	Responsável
01	Owner Técnico, Owner de Negócio, Data Steward e Data Custodian definidos	Engenheiro + Arquiteto
02	Classificação de dados definida (Público/Interno/Restrito/Confidencial/Crítico)	Data Steward
03	Domínio e schema Bronze identificados (existente ou aprovação para novo)	Engenheiro de Dados
04	Contrato da fonte documentado (schema, frequência, volume, formato)	Engenheiro de Dados
05	Estratégia de evolução de schema definida (aditiva/breaking — ver seção 5.4)	Engenheiro + Data Steward
06	Diretório {produto}/landing/{domínio}/ e {produto}/raw/{domínio}/ criados	Engenheiro de Dados

#	Item	Responsável
07	Schema Bronze criado ou validado no catálogo do produto	Engenheiro de Dados
08	Service principal de ingestão configurado com credencial no Key Vault do produto	Arquiteto de dados
09	Pipeline de ingestão implementado e testado (idempotente, com detecção de mudança de schema)	Engenheiro de Dados
10	Tabela Bronze criada, schema validado, amostra conferida	Engenheiro de Dados
11	Metadados de negócio registrados no Unity Catalog (descrição, CDEs, glossário)	Data Steward
12	Regras de qualidade Gold documentadas e aprovadas pelo Owner de Negócio	Engenheiro + Owner
13	Pipeline Silver implementado, testado e documentado com tratamento de schema evolution	Engenheiro de Dados
14	Política de Time Travel configurada por camada conforme seção 5.5.1	Data Custodian
15	Entrada no runbook do produto (re-ingestão, alertas, SLA, procedimento de backfill)	Engenheiro de Dados
16	Acesso dos grupos corretos configurado no Unity Catalog	Arquiteto de Dados

10.3 Observabilidade e Monitoramento

- ❑ **Logs estruturados:** Todo pipeline gera log JSON com: job_name, produto, run_id, run_type (regular/backfill), start_time, end_time, records_read, records_written, status, error_message, schema_version.
- ❑ **Métricas de qualidade:** Pipelines Silver registram: % nulos por CDE, volume descartado, violações de regra de qualidade.
- ❑ **Alertas:** Jobs críticos com alertas no Databricks para falhas e SLA excedido.
- ❑ **Dashboard operacional:** Status consolidado de jobs por produto, volume de dados por camada, alertas ativos e métricas de Time Travel (versões retidas por tabela).

11. Exemplo Prático — Produto SIADS / Domínio Almoxarifado

11.1 Contexto

Campo	Valor
Produto	SIADS (Sistema Integrado de Administração de Serviços)

Campo	Valor
Origem	Sistema SIAFEM — extração diária de movimentações de almoxarifado
Frequência	Carga incremental diária
Owner Técnico	Engenheiro João de Deus
Owner de Negócio	Coordenação de Logística / SEGES / Maria das Graças
Data Steward	Analista Sênior de Dados SIADS/ João dos Anjos
Data Custodian	Arquiteto de Plataforma CDATA/ Maria Clara
Classificação	Restrito
Resource Group	rg-cdata-siads
Produto Gold	Painel de Indicadores de Almoxarifado

11.2 Estrutura ADLS

```
siads/landing/almoxarifado/carga_20240115_001/almoxarifado_20240115_001.parquet
siads/raw/almoxarifado/2024/01/15/almoxarifado_20240115_001.parquet
siads/bronze/almoxarifado/2024/01/15/almoxarifado_20240115_001.parquet
siads/curated/material_consumo/ <- Delta com Time Travel x dias
siads/serving/indicadores_almox/ <- Delta com Time Travel x dias
siads/logs/carga_almoxarifado/2024/01/15/run_20240115_001.json
siads/checkpoints/stream_almox/
```

*Revisar se parquet ou Delta External Table para bronze

11.3 Estrutura Databricks

```
siads
├── 001_bnz_siafem
│   └── almoxarifado <- colunas + _ingestion_date, _source_file, _run_id,
│       _schema_version
├── 002_slv_material_consumo
│   └── itens <- id_mov (PK), dt_referencia, descricao_padronizada,
│       saldo_calculado
└── 003_gld_indicadores
    ├── painel_almox <- competencia, consumo_total, giro_estoque,
    alerta_ruptura
    └── vw_estoque_atual <- view para Power BI
```

11.4 Fluxo Bronze → Silver → Gold

Etapa	Job	Origem → Destino	Transformação Principal
1 — Chegada	siads_almoz	landing/ → raw/	Sem transformação
2 — Ingestão	ingest_siads_almoz	raw/ → 001_bnz_siafem.almozarifado	Extração incremental, adição de metadados técnicos, detecção de schema evolution
3 — Refinamento	transform_siads_material_consumo	001_bnz_siafem → 002_slv_material_consumo.ite ns	Tipagem, deduplicação, padronização, lookup de referências em 000_ref_global
4 — Publicação	publish_siads_indicadores	002_slv_material_consumo → 003_gld_indicadores.painel_al moz	Agregação mensal, cálculo de giro de estoque, alertas de ruptura

11.5 Controle de Acesso — Produto SIADS

Identidade	Acesso	Escopo
rg-cdata-siads	Resource Group	Todos os recursos Azure do SIADS no ambiente DEV
sp_siads_ingest	WRITE landing/ + raw/ + Bronze	siads/landing/, siads/raw/, siads/bronze/ siads.001_bnz_siafem
sp_siads_transform	READ Bronze + WRITE Silver	siads.001_bnz_siafem (leitura) + siads.002_slv_* (escrita)
sp_siads_publish	READ Silver + WRITE Gold	siads.002_slv_* (leitura) + siads.003_gld_* (escrita)
sp_shared_write	WRITE Shared	shared/ + shared.000_ref_*
grp_data_engineers_siads	READ + WRITE + CREATE (Bronze, Silver, Gold)	Criação e manutenção de objetos em siads.001_bnz_*, siads.002_slv_* e siads.003_gld_*. Leitura em siads.000_ref_*.
grp_data_stewards_siads	READ todos schemas	Todos os schemas siads.* para curadoria de metadados
grp_data_analysts_siads	READ Silver + Gold WRITE Sandbox	siads.002_slv_* e siads.003_gld_* e sandbox do produto
grp_data_scientists_siads	READ Silver + WRITE Sandbox	Leitura em siads.002_slv_* e experimentação em sandbox

12. Recomendações Finais

12.1 Governança do Próprio Padrão

- ☐ Este documento deve ser versionado em repositório Git com revisão via Pull Request para qualquer alteração.
- ☐ Revisão semestral obrigatória com participação de arquitetos e engenheiros sênior.
- ☐ Novos produtos devem ser aprovados antes de receber container e catálogo — o nome curto do produto é imutável após criação.
- ☐ Todo time que inicia trabalho na plataforma recebe treinamento formal neste padrão.

12.2 Fases de Implementação

- ☐ Fase 1 — Imediato: Criar containers ADLS por produto com 8 pastas funcionais. Criar Resource Groups. Criar catálogos com schema 000_ref_*. Configurar grupos e service principals. Documentar runbooks iniciais.
- ☐ Fase 2 — 30 dias: Migrar datasets piloto existentes para nova estrutura. Implementar logs estruturados com run_type e schema_version. Criar checklist formal de onboarding de fonte. Configurar Time Travel por camada.
- ☐ Fase 3 — 90 dias: Implementar dashboard operacional consolidado. Executar primeiro backfill formal com procedimento documentado. Registrar lições aprendidas para versão 4.0 do padrão.

12.3 Evolução Tecnológica Recomendada

- ☐ Avaliar adoção do Delta Live Tables (DLT) para pipelines declarativas com schema evolution nativa
- ☐ Avaliar integração de catálogo de metadados de negócio complementar ao Unity Catalog para gestão de glossário e CDEs em escala

Quadro-Resumo Final — Elementos e Padrões

Elemento	Padrão	Exemplo	Observação
Container ADLS	{produto}	siads, pgd, ouvidoria	Um container por produto
Pasta funcional (nível 1)	{funcao}	landing, raw, bronze, curated, serving, logs, checkpoints, sandbox	8 pastas fixas por container
Dir. nível 2 (ADLS)	{dominio}	almoxarifado, metas	Domínio temático
Catálogo Databricks	{produto}	siads, pgd	Produto = fronteira de isolamento
Schema Referências	000_ref_{escopo}	000_ref_global	Referências e glossários
Schema Bronze	001_bnz_{sistema_origem}	001_bnz_siafem	Por sistema de origem
Schema Silver	002_slv_{dominio}	002_slv_material_cons umo	Por entidade/domínio consolidado
Schema Gold	003_gld_{produto_dados}	003_gld_indicadores	Por produto de dados/caso de uso
Tabela	{entidade}	almoxarifado, contratos	Snake_case, sem prefixo
View	vw_{entidade}_{escopo}	vw_estoque_atual	Prefixo vw_ obrigatório
Volume	vol_{tipo}_{conteudo}	vol_glossario_siads	Prefixo vol_
Resource Group Azure	rg-cdata-{produto}-{env}	rg-cdata-siads-dev	Um por produto por ambiente
Job/Pipeline	{produto}_ {dominio<opicional>}{acao}_job	siads_almox_ingest_job	Ação: ingest/transform/publish
Notebook	{acao}_{produto}_{entidade}	transform_siads_materi ais	Sem numeração forçada
Checkpoint	chk_{job_name}	chk_stream_siads_alm ox	Um por job de streaming
Arquivo	{entidade}_{YYYYMMDD}_{ seq}.{ext}	almox_20240115_001. parquet	Data+seq = unicidade
Service Principal	sp_{produto}_{acao}	sp_siads_ingest	Um por produto e ação

Elemento	Padrão	Exemplo	Observação
Grupo de Acesso	grp_{papel}_{produto}	grp_data_analysts_siads	Grupos por papel + produto
Regra geral	minúsculas + underscore	sem acentos, espaços, hífen	Padrão universal obrigatório

Anexo A — Resumo Executivo

Os dez pilares da arquitetura de dados multiproduto da CDATA — MGI/SEGES:

#	Pilar	Descrição
01	Arquitetura Lakehouse Multiproduto	O CDATA opera múltiplos produtos independentes. A plataforma foi desenhada para servir todos eles com infraestrutura compartilhada e isolamento garantido por container ADLS e catálogo Databricks por produto.
02	Container ADLS = Produto	Cada produto possui seu próprio container ADLS, com 8 pastas funcionais internas. Resource Group Azure por produto agrupa todos os recursos do produto para RBAC, billing e auditoria.
03	Catálogo = Produto no Databricks	Cada produto possui seu próprio catálogo Unity Catalog, garantindo fronteiras de acesso, linhagem e governança estruturalmente independentes.
04	Landing como Decisão Arquitetural	Landing é área física de quarentena exclusivamente no ADLS. Sem representação no Unity Catalog por design — dados não-validados não geram linhagem no catálogo.
05	Schema = Camada + Domínio	Schemas identificam a camada (000_ref_*, 001_bnz_*, 002_slv_*, 003_gld_*) e o domínio temático, com ordenação visual que reflete o fluxo de dados.
06	Três Camadas Progressivas	Bronze preserva o dado original; Silver entrega a verdade única limpa; Gold publica produtos de dados prontos para consumo. Progressão unidirecional e obrigatória.
07	Shared/ com Ownership Explícito	Dados compartilhados em shared/global/ têm service principal dedicado (svc_{produto}_shared_write), aprovação formal de Arquiteto, e linhagem registrada no Unity Catalog via schema 000_ref_*.
08	Quatro Papéis Formais de Governança	Owner Técnico, Owner de Negócio, Data Steward (qualidade semântica e CDEs) e Data Custodian (infraestrutura e conformidade) — todos obrigatórios para promoção Bronze→Silver.
09	Reprodutibilidade e Time Travel	Bronze imutável + Time Travel Delta em Silver (90 dias) e Gold (180 dias) + procedimento formal de backfill garantem auditabilidade completa para ambiente governamental.
10	Schema Evolution Gerenciada	Mudanças aditivas absorvidas automaticamente; mudanças breaking seguem processo de aprovação com versionamento de schema em Bronze e mapeamentos explícitos em Silver.

Anexo B — Checklist de Validação da Arquitetura

Utilize este checklist para confirmar conformidade com o padrão v3.0.

#	Dimensão	Critério de Validação	Status
01	Containers ADLS	Containers criados por produto ({produto}) com as 8 pastas funcionais internas?	☐ OK ☐ N/A
02	Resource Groups	Resource Group rg-cdata-{produto}-{env} criado por produto?	☐ OK ☐ N/A
03	Landing explícito	Landing tratada como área de quarentena física sem representação no Unity Catalog?	☐ OK ☐ N/A
04	Catálogos Databricks	Catálogos criados com padrão {produto}?	☐ OK ☐ N/A
05	Schemas Referências	Schema 000_ref_global e 000_ref_{dominio} criados conforme necessidade?	☐ OK ☐ N/A
06	Schemas Bronze	Schemas Bronze seguem 001_bnz_{sistema_origem}?	☐ OK ☐ N/A
07	Schemas Silver	Schemas Silver seguem 002_slv_{dominio}?	☐ OK ☐ N/A
08	Schemas Gold	Schemas Gold seguem 003_gld_{produto_dados}?	☐ OK ☐ N/A
09	Nomenclatura geral	Todos os objetos usam snake_case, minúsculas, sem acentos?	☐ OK ☐ N/A
10	Papéis formais	Owner Técnico, Owner de Negócio, Data Steward e Data Custodian atribuídos a cada dataset?	☐ OK ☐ N/A
11	Classificação de dados	Todos os datasets classificados (Público/Interno/Restrito/Confidencial/Crítico)?	☐ OK ☐ N/A
12	Metadados de negócio	Descrição funcional, CDEs e glossário registrados no Unity Catalog pelo Data Steward?	☐ OK ☐ N/A
13	Shared/ com ownership	sp_{produto}_shared_write configurado? Processo de aprovação para shared/global/ definido?	☐ OK ☐ N/A
14	Linhagem de shared/	Tabelas de referência registradas em 000_ref_* com linhagem visível no Unity Catalog?	☐ OK ☐ N/A
15	Schema evolution	Estratégia de schema evolution documentada por fonte (aditiva/breaking)?	☐ OK ☐ N/A
16	Time Travel Silver	Retenção Delta Silver configurada para 90 dias?	☐ OK ☐ N/A
17	Time Travel Gold	Retenção Delta Gold configurada para 180 dias?	☐ OK ☐ N/A
18	Procedimento backfill	Procedimento formal de backfill documentado no runbook do produto?	☐ OK ☐ N/A
19	Grupos AAD por produto	grp_data_engineers, grp_data_analysts e grp_data_stewards criados por produto?	☐ OK ☐ N/A
20	Service Principals	sp_{produto}_ingest, _transform, _publish e _shared_write criados?	☐ OK ☐ N/A

#	Dimensão	Critério de Validação	Status
21	Key Vault por produto	Credenciais no Key Vault do Resource Group do produto — não em código?	[] OK [] N/A
22	Sandbox isolada	Sandbox isolada de raw/, curated/ e serving/ — sem dados confidenciais?	[] OK [] N/A
23	Logs estruturados	Pipelines geram logs JSON com job_name, run_type, schema_version, status e volumes?	[] OK [] N/A
24	Checklist onboarding	Checklist de nova fonte (16 itens) executado com ao menos uma fonte real por produto?	[] OK [] N/A
25	Treinamento	Times dos produtos treinados no padrão v3.0 antes de iniciar implementações?	[] OK [] N/A

Anexo C — Glossário de Siglas

Sigla	Extensão	Definição
AAD	Azure Active Directory	Serviço de identidade da Microsoft usado para gerenciar grupos de acesso e service principals da plataforma.
ACID	Atomicity, Consistency, Isolation, Durability	Propriedades que garantem confiabilidade de transações em banco de dados, implementadas pelo Delta Lake.
ADLS Gen2	Azure Data Lake Storage Generation 2	Serviço de armazenamento em nuvem da Microsoft otimizado para cargas analíticas de grande volume. Substrato físico da plataforma.
API	Application Programming Interface	Interface de programação usada como fonte de dados na ingestão e como consumidor de dados da camada Gold.
BI	Business Intelligence	Ferramentas de inteligência de negócios, como o Power BI, que consomem dados da camada Gold da plataforma.
CDATA	Coordenação de Dados	Equipe responsável pela plataforma de dados que serve múltiplos produtos do governo federal.
CDE	Critical Data Element	Elemento de dado crítico; colunas de maior impacto para decisões de negócio ou conformidade regulatória, registradas no Unity Catalog.
dbt	Data Build Tool	Ferramenta open-source para transformações analíticas, avaliada como evolução tecnológica para as camadas Silver e Gold.

DEV	Development	Ambiente de desenvolvimento. Único ambiente ativo no escopo do Padrão de Arquitetura v3.0.
DLT	Delta Live Tables	Recurso do Databricks para pipelines declarativas com schema evolution nativa, recomendado para evolução futura da plataforma.
IAM	Identity and Access Management	Gerenciamento de identidade e acesso no Azure. Define quem pode fazer o quê nos recursos da plataforma.
LGPD	Lei Geral de Proteção de Dados	Legislação brasileira que regula o tratamento de dados pessoais, influenciando classificações e controles de acesso na plataforma.
MFA	Multi-Factor Authentication	Autenticação multifator, exigida para acessos classificados no nível Crítico.
MGI	Ministério da Gestão e da Inovação em Serviços Públicos	Órgão federal responsável pela gestão pública e inovação, cliente da plataforma CDATA.
PGD	Programa de Gestão e Desempenho	Produto/sistema de informação do governo federal hospedado na plataforma CDATA, com domínios de metas, entregas, servidores e avaliação.
PII	Personally Identifiable Information	Informação Pessoal Identificável. Dados que permitem identificar um indivíduo, sujeitos às regras da LGPD.
PRD	Production	Ambiente de produção, referenciado como expansão futura da arquitetura.
RBAC	Role-Based Access Control	Controle de acesso baseado em papéis, utilizado para gerenciar permissões no Azure e no Unity Catalog.
SEGES	Secretaria de Gestão e Inovação	Unidade do MGI, parte do contexto organizacional da arquitetura de dados.
SIADS	Sistema Integrado de Administração de Serviços	Produto do governo federal com domínios como almoxarifado, contratos e patrimônio, hospedado na plataforma CDATA.
SIAFEM	Sistema Integrado de Administração Financeira para Estados e Municípios	Sistema de origem de dados para o produto SIADS.
SLA	Service Level Agreement	Acordo de nível de serviço. Prazo e disponibilidade garantidos para entrega do dado, obrigatório nas camadas Silver e Gold.

SP	Service Principal	Identidade de serviço no Azure usada por pipelines e jobs para acessar recursos sem interação humana.
-----------	-------------------	---

Anexo D — Glossário de Termos

Termo	Definição
Backfill	Reprocessamento de um intervalo histórico completo de dados. Executado com flag --backfill=true, ativando idempotência por hash de linha e desativando deduplicação por data.
Bronze (001_bnz_*)	Camada de fidelidade à fonte. Preserva o dado original acrescido de metadados técnicos (data de ingestão, hash, run_id). Imutável após ingestão. Acessível apenas por pipelines Silver.
Checkpoint	Estado persistido de um job Spark Structured Streaming para tolerância a falhas e retomada. Um diretório por job na pasta checkpoints/ do produto.
Container	Unidade organizacional primária do ADLS Gen2. Cada produto/cliente possui um container próprio, que é a fronteira de controle de acesso, lifecycle e billing.
Curated	Pasta funcional do ADLS correspondente à camada Silver. Dados em formato Delta com Time Travel habilitado. Zona de trabalho analítico principal.
Data Custodian	Responsável pela custódia técnica da infraestrutura: configuração de permissões, rotação de secrets, políticas de retenção, monitoramento de acesso e conformidade com a LGPD.
Data Steward	Responsável pela qualidade semântica do dado: define CDEs, mantém o glossário de negócio e valida que metadados estão corretos e atualizados no Unity Catalog.
Delta Lake	Formato de armazenamento transacional open-source que adiciona ACID, Time Travel, schema enforcement e otimizações de leitura sobre arquivos Parquet. Obrigatório em Silver e Gold.
Domínio temático	Agrupamento semântico de entidades de negócio relacionadas dentro de um produto. Exemplos: almoxarifado, contratos e patrimônio no SIADS.
Glossário de negócio	Volume dedicado (vol_glossario_{produto}) que centraliza os termos de negócio do produto, mantido pelo Data Steward e versionado com aprovação do Owner de Negócio.

Gold (003_gld_*)	Camada de produto de dados. Dados prontos para BI, APIs e dashboards, contendo agregações, KPIs e modelos dimensionais. SLA definido. Time Travel de 180 dias.
Idempotência	Propriedade dos pipelines em que a reexecução com as mesmas entradas produz o mesmo resultado, sem duplicar ou corromper dados.
Key Vault	Serviço Azure para armazenamento seguro de credenciais e secrets. Credenciais de service principals são armazenadas no Key Vault do Resource Group do produto.
Lakehouse	Paradigma arquitetural que combina a flexibilidade de um Data Lake com as capacidades de governança e desempenho de um Data Warehouse. Base da arquitetura da plataforma.
Landing	Área de recebimento temporário exclusivamente física no ADLS. Sem representação no Unity Catalog. Recebe arquivos externos antes de qualquer validação. Retenção de 7 dias.
Linhagem	Rastreabilidade automática registrada pelo Unity Catalog entre tabelas produtoras e consumidoras, permitindo análise de impacto e identificação de dependências.
mergeSchema	Opção do Delta Lake (mergeSchema=true) que permite a absorção automática de novas colunas adicionadas pela fonte na ingestão Bronze.
Owner de Negócio	Responsável pela definição das regras de negócio, aprovação da promoção Bronze→Silver e aceitação formal do dado Gold. Geralmente a área gestora do sistema.
Owner Técnico	Responsável pela pipeline, qualidade técnica e SLA de entrega do dado. Geralmente o engenheiro de dados do produto. Obrigatório para promoção Bronze→Silver.
Pasta funcional	Uma das oito subpastas dentro de cada container de produto (landing, raw, curated, serving, shared, logs, checkpoints, sandbox), que organizam dados por propósito.
Raw	Pasta funcional do ADLS que armazena os dados brutos exatamente como recebidos da fonte. Imutável após ingestão. Mapeada como tabelas Delta external no schema Bronze.
Referências (000_ref_*)	Schemas que agrupam entidades compartilhadas entre múltiplos catálogos, como tabelas mestres, taxonomias e classificações oficiais. O prefixo 000 garante ordenação visual antes das camadas de dados.
Resource Group	Agrupamento lógico de recursos Azure por produto e ambiente (rg-cdata-{produto}-{env}). Facilita RBAC, billing, auditoria e offboarding de produto.

Runbook	Documento operacional do produto que descreve fontes, frequência de carga, owners, procedimentos de re-ingestão, alertas, SLAs e pontos de contato.
Sandbox	Área isolada de experimentação individual dentro do container do produto. Sem SLA, sem dados confidenciais ou críticos. Somente para exploração controlada.
Schema Bronze	Registro da estrutura de colunas da fonte no momento da ingestão, incluindo a coluna <code>_schema_version</code> com hash do schema atual para garantir reprodutibilidade.
Schema evolution	Estratégia para absorver mudanças no schema de fontes externas (novas colunas, renomeações, remoções) sem quebrar pipelines downstream. Classificada em aditiva ou breaking.
Serving	Pasta funcional do ADLS correspondente à camada Gold. Delta otimizado para leitura por ferramentas de BI e APIs, com Time Travel de retenção estendida para auditoria.
Silver (002_slv_*)	Camada de verdade única. Dados limpos, deduplicados, tipados e padronizados. Referência para consumo técnico por analistas e cientistas de dados. Time Travel de 90 dias.
Soft delete	Funcionalidade do ADLS que retém arquivos excluídos por um período (7 dias em DEV) antes da remoção permanente, protegendo contra deleções acidentais.
Time Travel	Capacidade do Delta Lake de acessar versões históricas de uma tabela, permitindo auditoria, reprocessamento e restauração de dados em qualquer ponto anterior no tempo.
Unity Catalog	Camada de governança e catalogação lógica do Databricks. Organiza catálogos por produto, schemas por camada e domínio, e registra linhagem de dados automaticamente.

Arquiteto Responsável: _____

Data de Validação: _____